SoDA
ODISSEI Social Data Science Team

# Future-proof your research workflow with Git

## An introductory workshop

ODISSEI Early Career Day

4 November 2026 · 13:45–16:00

# Workshop goals

By the end, you can:

- **Track** changes to a research project.
- **Recover** an earlier version when something goes wrong.
- **Synchronize** work between your computer and GitHub.
- **Work in branches** without disturbing the main version.
- **Propose, review, and merge** a colleague's changes.
- **Understand** how to further integrate Git into your (AI-assisted) research workflow.

> You do not need to write or run code to learn any of these.

# Agenda

| Time | Activity |
|---|---|
| 13:45–13:55 | Git, GitHub and research files |
| 13:55–14:15 | Repository setup and cloning |
| 14:15–14:40 | Changes, commits, history, push/pull |
| 14:40–14:50 | Break |
| 14:50–15:10 | Branches |
| 15:10–15:45 | Issues, pull requests and peer review |
| 15:45–16:00 | Git and AI-assisted research |

# Familiar research project problems

`analysis_final.R` → `analysis_final_v2.R` → `analysis_final_REALLY_FINAL.R`

Or:

- An earlier working script disappears after a modification.
- Two collaborators edit the same file and exchange copies by email.
- A result changes, but nobody remembers *which code changed*.

> **Git helps you make changes visible and reversible.**

## Add screenshot for git history

# Three different tools

| Tool | What it is | What we'll use it for |
|---|---|---|
| **Git** | Version-control system | Track project history |
| **GitHub** | Website for hosting Git repositories | Share, discuss, review changes |
| **GitHub Desktop** | Visual Git application | Work locally without commands |

**Git ≠ GitHub.** Git also works without GitHub.

> Add screenshots for GitHub Desktop and GitHub.com to show difference

# The mental model: two copies, one history

**Local repository** = project on your computer
**Remote repository** = shared copy on GitHub

```
Your computer                          GitHub
  [files + Git history]  --push-->  [files + Git history]
  [files + Git history]  <--pull--  [files + Git history]
```

- **Commit:** save a meaningful snapshot to local Git history.
- **Push:** publish local commits to GitHub.
- **Pull:** bring remote commits into your local branch.

# Exercise 1 · Your private repository (8 min)

1. Sign in at **GitHub.com** and choose **New repository**.
2. Name it `research-git-practice`.
3. Choose **Private** and **Add a README file**.
4. Click **Create repository**.
5. Check the **Private** label and open `README.md`.

**Goal:** You own a private repository on GitHub.

Add screenshot of GitHub repository creation page

> Add screenshot of GitHub repository page with README.md open

# A repository is more than a folder

A **repository** contains:

- Project files: code, documentation, configuration.
- A **history of commits**: what changed, when, and by whom.
- One or more **branches**: different lines of development.

# Exercise 2 · Clone it locally (6 min)

1. On GitHub, open **your** new repository.
2. Click **Code → Open with GitHub Desktop**.
3. Choose where to save the local repository.
4. Click **Clone**.
5. Open the cloned folder in your file manager.

**Check:** Can you find `README.md` on your computer?

> Add screenshot of GitHub Desktop clone dialog

> Add screenshot of local folder with README.md

# Two places

**On GitHub.com:** your project exists online.

**On your laptop:** the same files exist in a Git repository.

**In GitHub Desktop:** you can see your local repository.

# Three stages of a change

1. **Edit and save** a file → changed working copy
2. **Commit** in GitHub Desktop → recorded local snapshot with a message

> A commit should describe a coherent change, not just “stuff”.

3. **Push** in GitHub Desktop → available on GitHub

> Add screenshot of changing README.md in a text editor

> Add screenshot of GitHub Desktop showing the changed file in **Changes** view

> Add screenshot of GitHub Desktop showing the commit dialog with a message

# Read the diff before committing

**GitHub Desktop → Changes** shows:

- **Green / +:** added lines.
- **Red / −:** removed lines.
- File names: which files changed.

> Good commit message: "Document missing-data handling"
> Weak commit message: "updates"

Add screenshot of GitHub Desktop commit history

# Workshop research example

> What factors influence the number of citations a research publication receives?

**SORTD = SoDa Open Research Teaching Datasets**

OpenAlex-derived teaching data for exploring research publications.

> Add a QR-code to the dataset page

# Exercise 3 · Make your first commit (10 min)

1. Open `README.md` in a **plain-text editor**.
2. Add `## Research question` and one question about SORTD.
3. Record the SORTD source link and dataset version in `docs/methods.md`; save.
4. Return to GitHub Desktop → **Changes**.
5. Select README and methods; inspect the diff and describe the change.
6. Click **Commit to main**.

**Goal:** One new commit in your local history.

# Working with data files

> Create a data folder in your cloned repository `/data`
> Save the zip file to `/data`

> Add screenshot of the created data folder in the cloned repository
> Ask why the data folder is empty in GitHub Desktop

> Add screenshot of the zip file in the data folder in GitHub Desktop
> Ask why the content of the zip file is not visible in GitHub Desktop

# Plain text and binary files

| Plain text | Binary formats |
| --- | --- |
| `.md`, `.R`, `.py`, `.csv`, `.tex` | `.docx`, `.xlsx`, `.pdf`, images |
| Git can show changes line by line | Usually no useful line-by-line diff |
| Easier to review and merge | Often need the original application |

Git can store both. **Readable diffs make collaboration easier.**

> Do we want to commit the zip file? Why or why not?

# .gitignore

What is `.gitignore`?

# Exercise 5 · Thou shall not commit these files (6 min)

In your cloned folder, create `.gitignore`.

Put these lines in `.gitignore`:

```
/data/
/output/
.env
.DS_Store
```

> Add screenshot of the .gitignore file in GitHub Desktop

# Exercise 6 · Publish (5 min)

1. In GitHub Desktop, click **Push origin**.
2. Open the repository on GitHub.com.
3. Confirm the new README text is there.
4. Confirm the new commit appears in **History**.

> Add screenshot of the commit history on GitHub.com

# Exercise 7 · A remote edit and a pull (5 min)

1. On GitHub.com, open `README.md` and click the pencil.
2. Add `Workshop practice: 4 November 2026`.
3. Choose **Commit changes** directly to `main` for this solo exercise.
4. In Desktop, click **Fetch origin**, then **Pull origin**.
5. Open the local README and find the new line.

**Check:** The browser commit now appears in Desktop's History.

# Undoing changes

- **Discard uncommitted edit:**
  - Gone is gone
  - Discard selected changes only if you truly don't need them.
- **Revert previously committed change:**
  - Use **History → Revert Changes in Commit** to make a new commit that reverses it.

> Add screenshot of the revert dialog in GitHub Desktop

> Add screenshot of the commit history showing the revert commit

# Quiz · Which action?

1. Your edit is saved but absent from History. What comes next?
2. Your commit is local. How do you send it to GitHub?
3. Your colleague made some changes online. How do you update locally?
4. A committed change was wrong. How can you reverse it visibly?

# Quiz · Answers

1. **Review the diff and commit.**
2. **Push.**
3. **Fetch, then pull.**
4. **Revert the commit**, then push the new commit.

# Break · 14:40–14:50

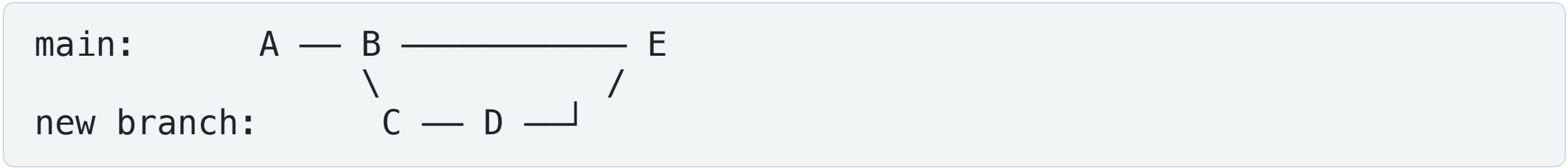
**Next:** Collaboration workflows with branches, issues, and pull requests.

> Add screenshot of the main branch in GitHub Desktop

# What is a branch?

A **branch** is a separate line of development in the *same* repository.

- `main` = the shared, stable line.
- `docs/add-reproducibility-note` = work in progress.
- Changes can be reviewed **before** being merged into `main`.




**Branch ≠ separate repository.**

Add screenshot of multiple branches in a project

# Branching in GitHub Desktop

1. Start from an up-to-date **main** branch.
2. Click **Current Branch → New Branch**.
3. Give it a descriptive name.
4. Edit, review, and **commit** as usual.
5. **Publish branch** to share it on GitHub.

**Switching branches** changes which version of your project you are working on.

> Add screenshot of the branch selection menu in GitHub Desktop

# Exercise 8 · Work on a branch (10 min)

1. In GitHub Desktop, select `main`; **Fetch/Pull** if updates are offered.
2. Create `docs/reproducibility-note`.
3. Open `docs/methods.md` and add one sentence about documentation.
4. Review the diff and **commit**.
5. Click **Publish branch**.
6. On GitHub, verify both branches exist.

**Quiz:** Does `main` contain your branch's new sentence yet?

# Quiz · Answer

**No.** Your change is on the new branch.

- A branch lets you develop a proposed change independently.
- The change is **not yet in** `main`.
- The branch is now visible on GitHub because you **published** it.

**How does a colleague decide whether to incorporate it?**

> Next: an issue and a pull request.

# Two GitHub collaboration tools

## Issue = work to be done

> “Document which observations are excluded from the analysis.”

## Pull request (PR) = proposed change to review and merge

> “I updated `docs/methods.md` to clarify exclusion rules.”

**Typical sequence:** Issue → branch → commits → PR → review → merge.

> Add screenshot of an issue and a pull request in GitHub

# Research collaboration scenario

### Person A = repository owner

- Opens a GitHub issue.
- Shares access with Person B to their `research-git-practice` repository.

### Person B = contributor / author

- Clones A's repository.
- Makes a change on a **new branch**.
- Opens a pull request.

### Person A = reviewer

- Reviews the proposal and merges it.

**Both:** Update the local `main` after the merge.

# Exercise 9A · Share and plan (7 min)

**Person A:**

1. Open a GitHub issue:

> "Add a sentence explaining how the analysis is documented."

2. On your repository: **Settings → Collaborators → Add people**.
3. Invite Person B's GitHub username.

**Person B:** Accept the invitation, then **clone A's repository**.

# Exercise 9B · Propose a change (10 min)

**Person B:**

1. On A's cloned repository, start from **main**.
2. Create branch `docs/clarify-workflow`.
3. Add a sentence to `docs/methods.md`.
4. Review the diff → **Commit** → **Publish branch**.
5. Click **Preview/Create Pull Request** in Desktop.
6. On GitHub, open a PR **into** `main`; mention the issue.

**Do not merge your own PR yet.**

# What should a reviewer check?

Before merging research code or documentation:

- **Purpose:** What is this change supposed to achieve?
- **Scope:** Were only the intended files edited?
- **Diff:** What lines were added or removed?
- **Evidence:** If code changed, was it run or tested?
- **Risks:** Any confidential data, passwords, or unintended outputs?

> A green “Merge” button is not proof of correctness.

# Exercise 9C · Review, merge, synchronize (8 min)

## Person A:

1. On GitHub, open B's PR → **Files changed**.
2. Leave a comment or **Approve** the change.
3. **Merge pull request** into `main`.

## Both people:

4. In GitHub Desktop, **switch to `main`**.
5. **Fetch origin → Pull origin** (when offered).
6. Confirm the new sentence appears locally.

# What happened?

Person B did **not** overwrite A's work.

Instead, B:

1. Proposed a change on a branch.
2. Made the diff visible in a pull request.
3. Received review.
4. Had the change merged into `main`.

**Quiz:** What can a pull request show that an emailed file cannot?

# Quiz · Answer

A pull request connects the proposed **diff** with:

- The reason for the change and its issue.
- Comments and review decisions.
- The commits and the eventual merge into `main`.

**Both collaborators then pull the shared version.**

# Daily workflow for a research project

```
PULL latest main
      ↓
CREATE a branch for one coherent change
      ↓
EDIT → REVIEW DIFF → COMMIT
      ↓
PUSH / PUBLISH branch
      ↓
OPEN PR → DISCUSS / REVIEW → MERGE
      ↓
PULL updated main
```

You now know the essentials of this workflow.

# What belongs in a research Git repository?

## Usually yes

- Analysis scripts and notebooks (when appropriate)
- README and methods documentation
- Small configuration files and environment definitions
- Tests, codebooks, and **synthetic/example** data

## Usually no

- Confidential or personal research data
- API keys, passwords, credentials
- Large generated datasets or temporary output files

**Important:** Deleting a secret in a later commit does not reliably erase it from Git history.

# The five things to remember

1. **Commit**: record a meaningful local change.
2. **Push / Pull**: synchronize with GitHub.
3. **Branch**: keep new work separate from `main`.
4. **Pull request**: propose and review a change.
5. **History / Revert**: understand and recover prior work.

**Git supports reproducibility; it does not guarantee it.**

# Supercharge your research workflow

- AI-assisted coding
- AI-assisted writing

> Add screenshot of AI proposed changes

# AI-assisted changes in general

If an AI assistant makes any change to your file(s), you should:

- **Branch first** so the proposal is isolated.
- **Review the diff**: which files and lines changed?
- **Test / inspect outputs** before accepting the result.
- **Commit meaningful units** rather than a giant opaque change.
- **Revert** if the modification introduces problems.

Git records changes; **you** remain responsible for research validity.

# Git with your coding tools

## You learned Git through GitHub Desktop.

The same repository can also be managed using:

- VS Code's built-in Source Control interface
- RStudio's Git pane
- Git in a terminal

**Same concepts:** repo, commit, branch, push, pull, merge.

# Writing research papers with Git

The same workflow can version-control:

- LaTeX source (`.tex`) and bibliographies (`.bib`)
- Manuscript figures and tables generated by code
- Revisions, comments, and writing contributions

Can be especially useful for:

- **Git + AI integration**
- **Overleaf + Git integration**

Add screenshot of Overleaf Git integration

> Add screenshot of VS Code Git integration with AI-assisted writing

# Useful official guides

- Getting started with GitHub Desktop
- Cloning a repository
- Creating a PR from Desktop
- Inviting collaborators
- Reviewing pull requests
- Reverting commits
- Carpentries: GitHub without the command line

# Optional · When Git gets complicated

**Merge conflict** = two sets of changes cannot be reconciled automatically.

Common prevention habits:

- Pull before starting new work.
- Keep branches and pull requests focused.
- Coordinate when editing the same lines.
- Review conflicts carefully; never choose an option blindly.

There is no need to master conflict resolution today.

# What do the Git commands look like?

| GitHub Desktop action | Command-line equivalent |
|---|---|
| Clone | `git clone URL` |
| Review status | `git status` |
| Record a change | `git add …` then `git commit -m "…"` |
| Send changes | `git push` |
| Retrieve/merge remote changes | `git pull` |
| Create and switch branch | `git switch -c branch-name` |

**These are optional ways to perform the same tasks.**